

Принцип роботи алгоритму оптимізації

DeliveryIQ — оптимізація мультимодальних маршрутів доставки

Цей документ покроково описує, як застосунок перетворює адресу депо та список точок доставки на оптимізований маршрут (TSP) або план розвезення флотом (CVRPTW). Кожен крок прив'язаний до конкретного модуля у вихідному коді.

1. Загальний потік даних

Конвеєр складається з шести логічних етапів. Кожен етап має чітко окреслений вхід і вихід; проміжні дані кешуються у **st.session_state**, щоб уникнути повторних завантажень мережі OSM та повторних обчислень матриці.

Етап	Модуль	Результат
1. Геокодування	geocoder.py	(lat, lon) для кожної адреси
2. Завантаження мережі	graph_builder.py	MultiDiGraph OSM (LSCC-зрізаний)
3. Прив'язка до вузлів	graph_builder.py	node_id для депо та точок
4. Часові мітки ребер	graph_builder.py	{drive, bike, walk} — три графи
5a. TSP (1 T3)	route_solver.py	Замкнений маршрут + час
5b. VRP (флот)	vrp_solver.py	Маршрути по T3 + попередження
6. Візуалізація	visualizer.py	Folium-карта з AntPath

Архітектурне рішення: для оптимізації мережа завантажується через `cached_network_at(lat, lon, radius)`, а не за назвою міста. Інакше Nominatim може поставити мітку міста далеко від району доставки і всі адреси прив'яжуться до одного граничного вузла.

2. Підготовка дорожньої мережі

2.1. Завантаження OSM

Через бібліотеку OSMnx завантажується граф із **network_type="all"** — щоб одразу мати ребра пішохідних доріжок, велодоріжок і проїздів. Перед завантаженням викликається **_configure_osmnx_tags()**, щоб залишити модальні теги (*bicycle*, *motor_vehicle*, *cycleway*:* тощо) на ребрах — ці теги пізніше визначатимуть прохідність.

2.2. Зрізання до LSCC

Сирий граф може містити невеликі ізольовані компоненти (відрізані дороги, помилки OSM). Виклик **ox.truncate.largest_component(G_raw, strongly=True)** залишає лише найбільшу сильно зв'язну компоненту. Це гарантує, що від будь-якого вузла можна доїхати до будь-якого іншого — критично для подальшого Дейкстри.

Прив'язка координат до вузлів виконується **ВИКЛЮЧНО** після зрізання, бо інакше можна отримати `node_id`, якого немає у фінальному графі — `build_distance_matrix()` моді кине

RuntimeError.

2.3. Часові мітки + три модальні копії

Функція **add_travel_times(G)** створює три глибокі копії графа і для кожного режиму пропоставляє атрибут **travel_time** (секунди) на кожному ребрі. Непрохідні для режиму ребра отримують сентинельне значення **PENALTY = 1e9 s** — достатньо велике, щоб домінувати над будь-яким реальним шляхом, але скінченне, щоб TSP-розв'язувачі могли формально завершити маршрут.

Режим	Швидкість	Що блокується (PENALTY)
drive	30 км/год	footway, pedestrian, steps, motor_vehicle=no/private
bike	15 км/год	steps, bicycle=no/dismount, односторонній рух проти напрямку
walk	5 км/год (steps × 0.5)	foot=no, access=private

3. Оптимізація TSP (один транспортний засіб)

Завдання комівояжера: знайти такий замкнений порядок відвідання точок $\pi = (\text{depot}, \pi_1, \pi_2, \dots, \pi_n, \text{depot})$, який мінімізує сумарний час подорожі

$$\text{Cost}(\pi) = \sum \text{matrix}[\pi[i] \rightarrow \pi[i+1]] \text{ для } i = 0 \dots n$$

3.1. Матриця відстаней (Фаза 1)

Функція **build_distance_matrix(G, nodes)** застосовує алгоритм Дейкстри для кожної пари (src, dst) і повертає словник **{{src, dst}: travel_time_seconds}**. Перед розрахунком вузли дедуплікуються через *dict.fromkeys()*: дві адреси можуть прив'язатися до одного OSM-вузла, і без дедуплікації TSP отримав би 0-вартісні ребра й нульовий сумарний час.

Якщо шляху немає (NetworkXNoPath), у клітинку записується PENALTY.

3.2. Гібридна 'остання миля' для drive

Багато адрес (під'їзди, дворики) недоступні для авто. Замість відкидати їх, застосунок паркує авто у **nearest_car_accessible_node()** — найближчому вузлі, інцидентному ребру з motor_vehicle-доступом — і додає пішохідний сегмент **walk_time = gap_m / walk_speed**, якщо gap_m ≤ 100 м. Інакше точка позначається як недоступна для авто.

3.3. Аудит досяжності

audit_reachability(matrix, nodes, labels) сканує матрицю на PENALTY-клітинки і повертає список об'єктів **UnreachableStop** з полями: вузол, мітка ('Stop #2'), список пунктів, з яких/до яких неможливо дістатися. UI рендерить ці об'єкти як іменовані попередження.

3.4. Розв'язання TSP — автовибір методу

Функція **solve_tsp(nodes, matrix, method="auto")** обирає стратегію залежно від розміру задачі:

Кількість точок	Метод	Складність
1–2	Nearest-Neighbour (NN)	$O(n^2)$
3–20	NN + 2-opt	$O(n^3)$ на ітерацію
21+	Genetic Algorithm (OX)	$O(P \cdot G \cdot n)$

Чому 2-opt замість Christofides на малих інстансах: Christofides потребує повного зв'язного неорієнтованого графа. Якщо хоч одна пара має PENALTY-вагу, допоміжний граф розсипається на компоненти і алгоритм падає. 2-opt же коректно опрацьовує PENALTY — він просто ніколи не вибере таке ребро, якщо є скінченна альтернатива.

3.5. Покрокова робота 2-opt

2-opt починає з NN-маршруту й ітеративно інвертує пари ребер: для кожної пари індексів (i, j), $1 \leq i < j < n$, формує кандидата

```
candidate = route[:i] + route[i:j+1][::-1] + route[j+1:]
```

Якщо **cost(candidate) < cost(best)**, замінює best і продовжує. Алгоритм зупиняється, коли жодна інверсія не покращує вартість (локальний оптимум) або досягнуто 2000 ітерацій. Депо зафіксоване на позиції 0 і $n-1$, тому ніколи не свопається.

3.6. Покрокова робота генетичного алгоритму

Хромосома — перестановка індексів точок (без депо). Депо додається на декодуванні: `route = [depot] + chrom + [depot]`.

- **Ініціалізація:** `populations_size=120` випадкових перестановок, плюс один NN-сід.
- **Селекція:** топ-10% популяції стають елітою.
- **Order Crossover (OX):** копіюється випадковий зріз `p1[a:b]` у дитину; решта позицій заповнюється генами `p2` у тому порядку, у якому вони з'являються у `p2` (без дублювань).
- **Swap-мутація:** з імовірністю `mutation_rate=0.02` міняються значення у двох випадкових позиціях.
- **Fitness:** $1 / (\text{cost} + 1)$ — ε уникає ділення на нуль для PENALTY-маршрутів.
- **Покоління:** 400 (за замовчуванням), потім обирається найкраща хромосома.

3.7. Реконструкція повного маршруту

TSP повертає лише послідовність точок зупинок. Щоб намалювати полілінію на карті, потрібен повний шлях по OSM-вузлах. **reconstruct_full_route(G, tsp_route)** для кожної пари (`stop[i]`, `stop[i+1]`) запускає Дейкстру і склеює сегменти, відкидаючи дублікати на стиках.

4. Оптимізація VRP (флот, CVRPTW)

Capacitated Vehicle Routing Problem with Time Windows: розподілити n точок між K транспортними засобами так, щоб мінімізувати

$$F = A \cdot T_{\text{total}} + B \cdot K + P \cdot |U|$$

де T_{total} — сума часів усіх маршрутів, K — кількість активних ТЗ, U — множина незамаршрутизованих точок, $P \geq A, B$. Обмеження: вантажопідйомність Q_{θ} та часові вікна $[tw_{\text{open}}, tw_{\text{close}}]$ кожної точки.

4.1. Фаза 1 — призначення режимів

`_assign_stops_to_modes(stops, fleet, depot, graphs):`

- Для кожної точки `_check_reachable()` запускає Дейкстру у кожному з трьох графів і повертає множину досяжних режимів.
- Серед сумісних режимів обирається той, де залишок сумарної місткості флоту максимальний.
- Залишок місткості зменшується на `weight_kg` точки.
- Точки, недосяжні жодним режимом або з вичерпаним флотом, потрапляють у список `unreachable`.

4.2. Фаза 2 — GA для кожної групи режиму

Для кожного пулу точок одного режиму викликається `_genetic_algorithm()` з параметрами тези:

Параметр	Значення	Призначення
POP_SIZE	50	Розмір популяції
N_GENERATIONS	150	Кількість поколінь
CROSSOVER_RATE	0.85	Імовірність кросоверу
MUTATION_RATE	0.15	Імовірність мутації
ELITE_SIZE	2	Скільки найкращих переходить без змін
TOURNAMENT_SIZE	3	Розмір турніру для селекції
GA_SEED	42	Детермінованість запуску

4.3. Декодер Split

Хромосома — це 'гігантський тур' (перестановка всіх клієнтів пулу, без розділювачів між машинами). Декодер послідовно проходить хромосому й додає клієнта до поточного маршруту, поки виконуються:

- **Capacity:** сума `weight_kg` $\leq$ `vehicle.capacity_kg`.
- **Time-window:** час прибуття $\leq$ `tw_close` клієнта (з урахуванням `service_time` попередніх).
- **Reachability:** жодне ребро маршруту не має `PENALTY`.
- **Round-trip:** після останнього клієнта ТЗ повертається у депо до `tw_close` депо.

Якщо клієнта не вдається додати, поточний маршрут закривається й береться наступний ТЗ із флоту. Невпихнуті клієнти потрапляють у **unassigned** і штрафуються через **UNROUTED_PENALTY = 1e6**.

4.4. Покоління GA

- **Турнірна селекція:** 3 випадкові — обирається найкращий.
- **ОХ-кросовер:** ймовірність 0.85.
- **Swap-мутація:** ймовірність 0.15.
- **Елітизм:** 2 найкращі без змін у наступне покоління.
- Найкраща хромосома за всі 150 поколінь декодується у фінальний розподіл маршрутів.

5. Формування результату

5.1. Об'єкти-носії результату

- **ModeResult** (TSP) — режим, послідовність зупинок, повний маршрут, `total_time_s`, легі (`LegInfo`).
- **VehicleRoute** (VRP) — ТЗ, зупинки, `tsp/full route`, `total_time_s`, `total_dist_m`, легі, `skipped_stops`.
- **LegInfo** — пара (`from`, `to`), `distance_m`, `travel_time_s`, `cumulative_time_s`. Використовується для побудови таблиці черговості та ETA.

5.2. Візуалізація

build_result_map() — TSP. AntPath кожного режиму (`drive` — червоний, `bike` — зелений, `walk` — синій), `LayerControl` для перемикання, депо — зелений маркер, точки — нумеровані червоні.

build_vrp_result_map() — VRP. Кольорова полілінія для кожного ТЗ (палітра `ColorBrewer Set1`, 10 кольорів), маркери з номером ТЗ.

6. Що бачить користувач

- Інтерактивна анімована карта з маршрутами.
- Картки метрик: загальний час `drive` / `bike` / `walk`.
- Таблиця черговості: від → до, відстань, час етапу, ETA.
- Банери попереджень: колізії вузлів, недосяжні точки, недоступні для авто, простоючі ТЗ.

Усі довгі обчислення кешуються у `st.session_state` з ключем (`depot.lat`, `depot.lon`, `radius`), тому повторне натискання 'Optimize Route' не запускає завантаження мережі OSM повторно.